

CSI247:
Data Structures

Lecture 04 – Java User Defined Classes

Department of Computer Science
B. Gopolang (247-275)



Previous Lesson ...

- Methods
  - Method Implementation
  - Method Invocation
  - Parameter passing
  - Variable scoping



Learning Objectives

By the end of this lesson, students should be able to :

- Differentiate between a class and an object
- Understand structure of a Java class
- Understand instance variables
- Understand and invoke different types of methods i.e
  - Constructors, void & value-returning methods
- Instantiate objects
- Write a User-defined class
- Test user-defined class methods

1. Overview of Object

- A Java program consists of interacting objects.

- An OBJECT
  - models a thing/entity
    - real or abstract
  - Has values (attributes) and operations (methods)
  - E.g. Student: attributes could be ID, name, DOB, etc

UNIVERSITY of BOTSWANA

THUTO KE THEBE

A CLASS in Java

- Block of code detailing:
  - How to create (instantiate) an object of the class
  - A collection of associated operations (methods)
    - i.e. a class defines objects and their operations
- Object: An instance of a class.
  - E.g. String str1= new String("Mpho");

```
// Object str1 is an instance of class String
```

2. Instantiating (Creating) an Object

- **Remember** : An object is an instance of a class.

- Syntax:

  Class_Type identifier = new Class_Type(actual_parameters);

- Example

  String courseName = new String("CSI241");

  Scanner input = new Scanner(System.in);

  PrintWriter oFile = new PrintWriter("outfile.txt");

  Dog d1 = new Dog("Fluffy", 1);

- We can also separate it into two lines like this:

```
Class_Type  identifier;
identifier = new Class_Type(actual_parameters);
```

Example

```
Scanner input;
input = new Scanner(System.in);
```

3. User Defined Class

- **Remember**: A class is used to define objects and associated operations.

- **Structure**

```
// import statements
public class ClassName {
    // instance variables (attributes)
    // constructors (special class methods )
    // other methods
}
```

4. Instance Variables

Instance Variable: Stores certain information about an object

- Object's attribute
- Declared on a class but outside a method.
- Accessible to all methods in the class.

- Note
  - Once created, each object has its own value for each instance variable.

Declaring Instance Variables

- Syntax (for primitive types)

  private type identifier;

  or

  private type identifier = value;

- Examples

  private boolean isSquare;

  private int count = 4;

- Syntax (for class types)

  private *Class_Type* identifier;

  or

  private *Class_Type* identifier = *new* *Class_Type*(actual_parameters);

- Examples

  private PrintWriter outputFile;

  private Scanner con = new Scanner(System.in);

5. Methods

Method: a definition of an action that the object can perform

- **Examples**
  - **String:**
    - equals(), length(), charAt(), indexOf(), substring(), etc
  - **Scanner:**
    - next(), nextLine(), nextInt(), nextDouble(), hasNext(), etc

Note:

- Objects interact with the environment via methods
  - Some methods **need** something from the environment
  - Some methods **give** something back to the environment

a) void methods (Modifiers)

They perform some action on the object
but **does not return a value**

- They often **change** or **display** values of instance variables.

i) Defining void methods

- Syntax

  public **void** *methodName(formal_Parameters)*

- Examples (from PrintWriter class)

  public void println(String str) {…} // Will accept 1parameter

  public void close() {…} *// Takes NO parameters*

  Note: *Methods accept different no. of parameters & data types*

ii) Invoking/using void Methods

- Syntax:

  objectName.methodName(actual_Parameters);

- Examples: (from PrintWriter class)

  Assumption: pw has been declared (and initialized) as a PrintWriter object

```
pw.println(str);        // str is a parameter of type String
pw.close();             // method requires no parameter
```

- **Please Note:**
  - Data type(s) of parameter(s) are omitted during invocation.

b) Value-returning methods (Accessors)

They may perform actions necessary to compute some value, but will return some value

- **Returns** a value to its environment

i) Defining value-returning methods

- Syntax

public ***returnType*** *methodName(formal_Parameters)*

- Examples

```
public int length() {...}          // from String class, no parameters
public char charAt(int index) {...}   // from String class, 1 parameter
public String nextLine() {...}  // from Scanner class
```

ii) invoking/calling value-returning methods

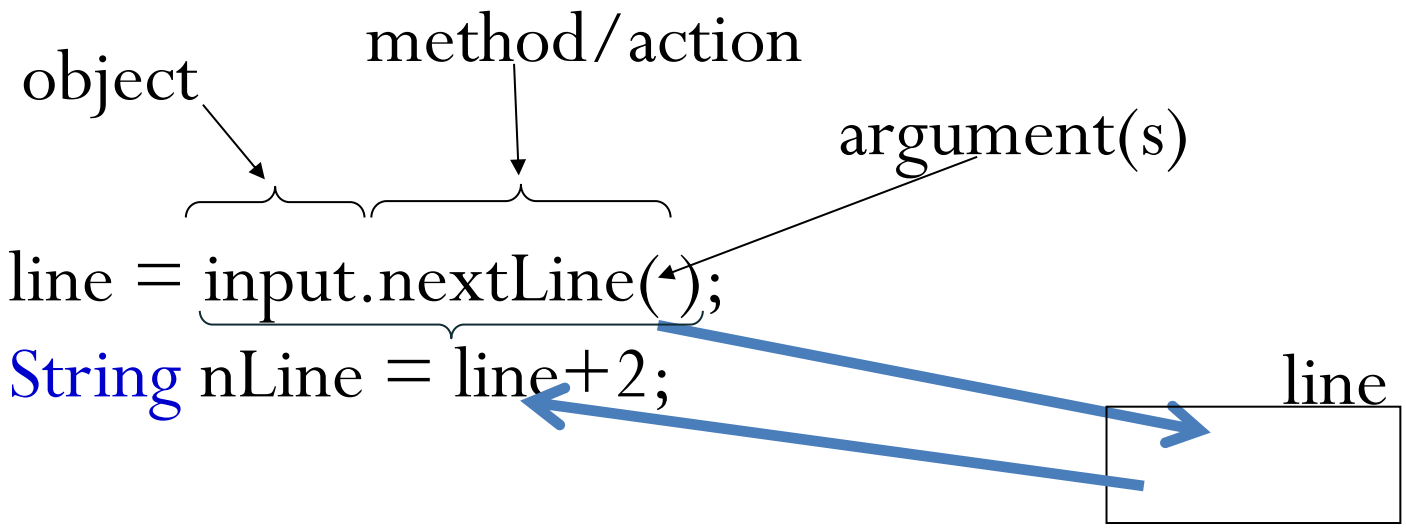
- Syntax:

  objectName.methodName(actual_Parameters)

- Used in expressions ONLY.
- Example

```
String line, inLowerCase;
while (input.hasNextLine() ) {    //assume input is a Scanner object
        line = input.nextLine();
        inLowerCase = line.toLowerCase();
        System.out.printf("Line in lowercase: %s\n", inLowerCase);
}
```

- Use the returned value anywhere a value of that type can be used.
- **Illustration (using code from previous slide):**



c) Constructors

Very special methods. Used to create class objects (using the new operator)

i) Purpose of a constructor:

- Initializes instance variables

  &

- Performs other necessary actions during object creation

ii) Properties of a constructor:

- Name is similar to class name
- No return type

iii) How many constructors per class?

- A class can have multiple constructors
  - They MUST share a name! (Similar to class name)

Homework

- Refer to Java API documentation to check number of constructors for these classes (and their differences):
  - String
  - Scanner
  - PrintWriter
  - FileReader

iv) Defining constructors

- **Syntax**

```
public ClassName(formal_Parameters) { //... }
```

- **Examples**

```
public String() {//...}
public Account(int newNumber, double newBalance) {
    //  ...
}
```

v) Using a constructor

- **Syntax**

Class_Type identifier = *new* *Class_Type*(actual_parameters);

Or

Class_Type identifier;

identifier = *new* *Class_Type*(actual_parameters);

- **Example**

Scanner scan = *new* ***Scanner***(System.in);



6. Writing a User-defined Class

Steps

- Identify:
  - Attributes (instance variables)
    - Appropriate name & type
  - Operations
    - Appropriate name
    - Parameter(s) and type(s)
    - Return types
- Then implement the class
  - Write code for the class
- Finally Test the class
  - Write an application class in which you invoke class methods



- Assuming **Account** class
  - Attributes
    - account number and balance
  - Account operations include
    - Changing account number
    - Changing balance (deposit or withdraw)
    - Retrieving account number
    - Retrieving account balance
    - Comparing two accounts
    - Etc
- We will use Account class to explain the rest of concepts in this handout.



a) Naming a class

- MUST start with an upper case letter
  - It is just a naming convention, but that MUST be followed.
  - Every new word in the name should start with an upper case letter.
- Pick a meaningful name.
- One class per file.

- Java **requires** that you name your file this way:

  className.java.

  Note: No Exceptions.

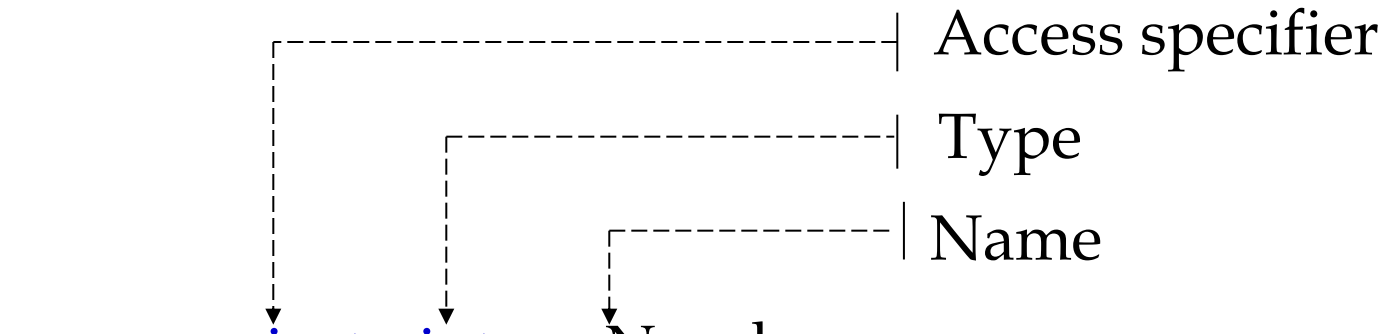
- Example:

```
public class Account{
 // ...
}
```

  - class MUST be saved as Account.java

b) Instance variables

- Generally, looks like this:



- Access specifier/Modifier:
  - Usually private
    - Only accessed by methods of their class (i.e. methods defined in the same class as the instance variable)
    - i.e. the public methods of their class
  - Why private? Later in the course!

c) Methods

- Methods are miniature programs
  - A class defines methods that can be applied to its objects
  - Objects are manipulated by invoking methods of its class.

- Syntax:

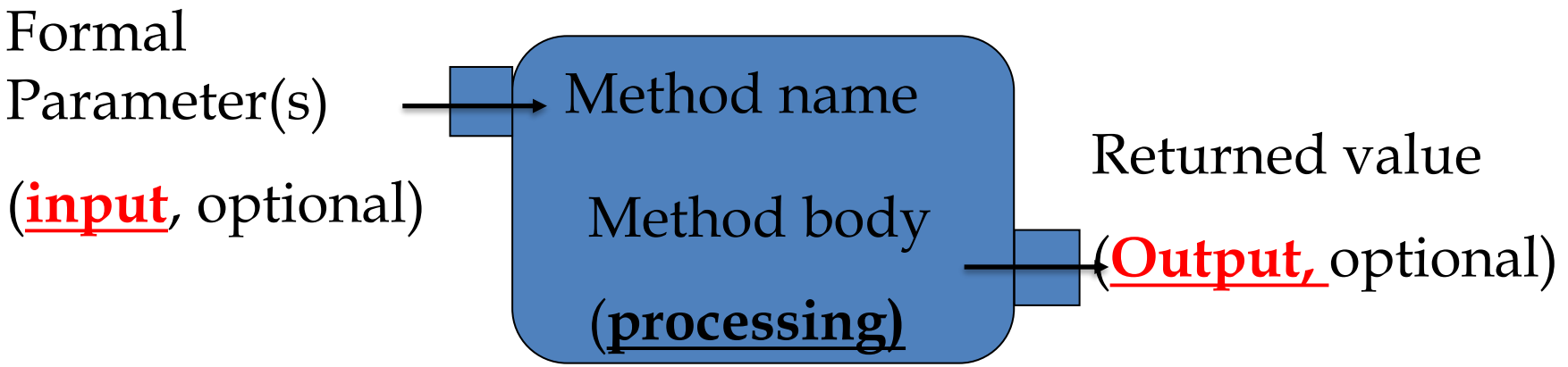
```
Access_specifier(s) returnType methodName(formalParameters){
    // Method's body: code to solve problem or answer question
}
```

Writing Methods

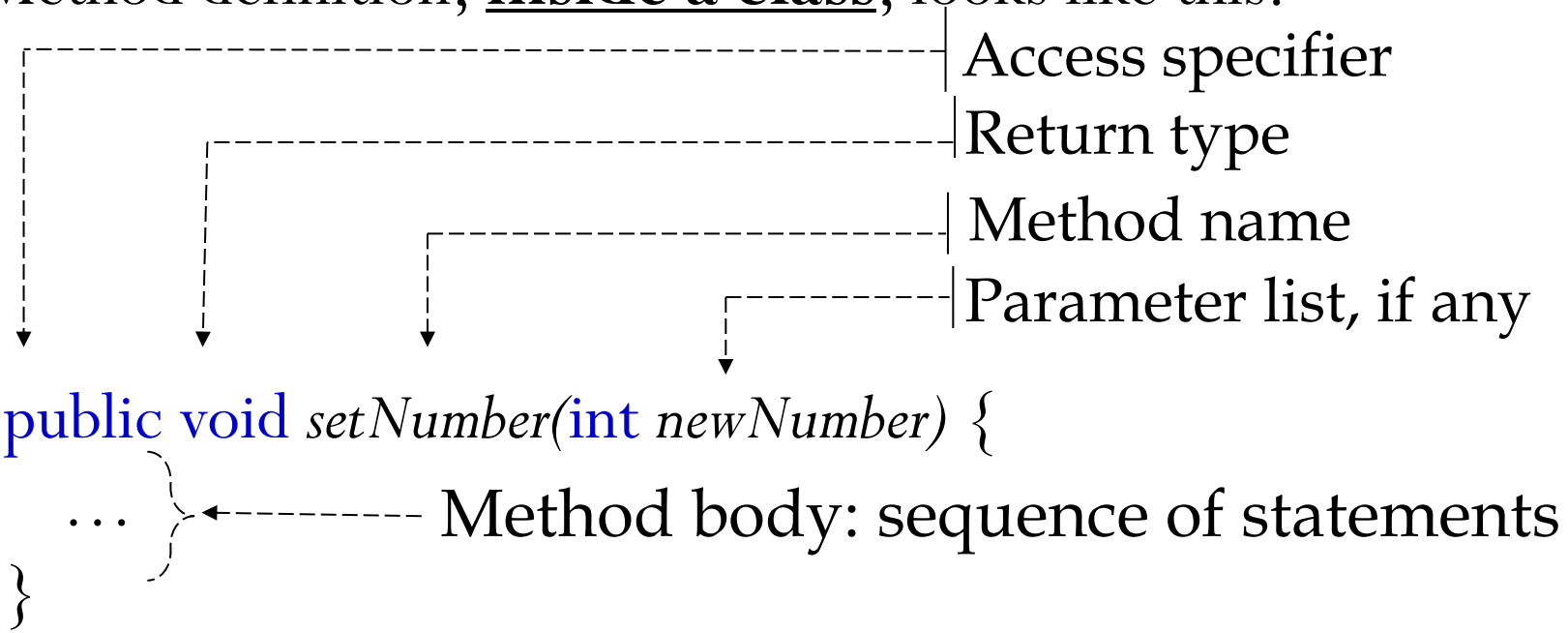
1. Determine method name:

   - starts with a lower case letter, should have a verb in it.

2. Determine formal parameters: input to the method
3. Determine return type (& what should be returned)
4. Write the method body: think problem solving template! **(input, processing, output)**



- Method definition, **inside a class**, looks like this:



WARNING!!!

- **Do not confuse method definition with method invocation**

d) Constructors

- Are used to initialize instance variables.
- Have the **SAME name as the CLASS**.
- **DO NOT** have a return type.
  - ***Class Account constructor***

```
public Account (int newNumber, double newBalance) {
    accNumber = newNumber;
    balance = newBalance;
}
```

PLEASE NOTE:

- If constructor is omitted in a class, Java adds the default constructor during class compilation!



Accounts class so far!

```
public class Account {
    // instance variables
    private int accNumber;
    private double balance;
    // constructor(s)
    public Account (int newNumber, double newBalance){
        accNumber = newNumber;
        balance= newBalance;
    }
    // other methods will go here
}
```

- **Default constructor**
  - Initializes attributes to "default" values.
    - int is 0, double is 0.0, char is '' & class types is null.

```
public Account() {
    accNumber = 0;
    balance = 0.0;
}
```

OR

```
public Account() {
    this (0, 0.0);
}
```

this —pointing to self

- A way for an object to refer to itself inside a class.
- Used as if it was the object's name.
- Used to distinguish between objects of the same class.

NOTE:

- For every instance variable (attribute) you need:
  - **Modifier**: method that changes the attribute's value
    - setter method
  - **Accessor**: method that accesses the object BUT does not change the attribute's value
    - getter method

e) Setter methods

- Used for changing values of instance variables

- **setNumber Method**:
  - Sets the value of the account number to the value of a given formal parameter

```
public void setNumber (int newNumber ) {
    accNumber = newNumber;
    // replaces accNumber with a new value
}
```

f) Getter methods

- Retrieves values of instance variables

- **<u>getNumber Method</u>**:
  - Retrieves the value of the account number.

```
public int getNumber( ) {
    return accNumber;          // retrieve accNumber and returns
    it to the place where this method is invoked
}
```

The return Statement

- Methods that return a value MUST have a return statement.
- **Syntax:**

  return *expression;*

  (expression's data type MUST match method's **returnType**)
- Example:

  public int **getNumber**( )
  - It **MUST** have a return statement as the last line in its body.
  - **Must** return an int value
    - Because that is the method's specified return type

g) equals method

- Used for testing equality between two objects of a similar type (accounts in this case)

```
// Assumption: accounts are equal if they have identical account
   numbers

public boolean equals(Account otherAcc) {
   return (accNumber == otherAcc.getNumber());
   //or return (this.getNumber() == otherAcc.getNumber());

}
```

h) toString method

- Return's a String representation of the object

```
public String toString() {
    String accString;          // local variable
    accString = "Acc No = " + accNumber + "Balance = " +
    balance;
    return accString;
}
```



Accounts class so far!

```
public class Account{
    // instance variables
    private int accNumber;
    private double balance;
    // constructors
    public Account() { //... }
    public Account (int newNumber , double newBalance) { //... }
    // other methods
    public void setNumber( int newNumber) {// ... }
    public int getNumber() { //... }
    public boolean equals(Account otherAcc) {// ... }
    public String toString() { //... }
}
```

7. Testing Account Class - Exercises

- Type the Account class as it appears in your lecture notes
  - Suggestion:
    - Start with the class skeleton, and compile it
    - Add the instance variables then compile
    - Add the methods, one at a time. Ensure you compile after adding each statement.
  - You MUST end up with a syntax error-free program

Question:

- Can we run the Account class program?
  - Answer & reason?

- We need to test the class to ensure its logic is correct
  - That requires a tester program!
- Using the Account class, write a Java program to perform the following:
  - Create two accounts, both with unknown values for account number and balance.
  - Change account number for both accounts to values read from the keyboard.
  - Display the account number for each account.
  - Display the message "Wow! Duplicate detected" if the two accounts are equal, otherwise "They are different!"

8. Testing class: Passing Parameters

```
…
acc1 Account = new Account();
int num = 76575;
acc1.setNumber( num);
…
```

Passing side (in file AccountTester.java)

- num: an int argument passed to setNumber

```
public class Account{
  … //constructors & some methods
  public void setNumber( int newNumber) {
    accNumber = newNumber;
  }
…  // remaining methods
}
```

Receiving side (in file Account.java):

- setNumber is defined to receive one int parameter.

IMPORTANT:

Type on the passing side & type on the receiving side **MUST** be compatible

Exercises – cont.

- Add the following methods to complete the Account class:
  - **setBalance**: changes balance to the specified one
  - **getBalance**: retrieves account balance
  - **equalBalance**: Method takes an account as its parameter, and returns true if the balance of the second account is equal to balance of the first account.
  - **withdraw**: Method takes amount as its parameter, and debit the account with that amount.
  - **deposit**: Method takes amount as its parameter, and credit the account with that amount

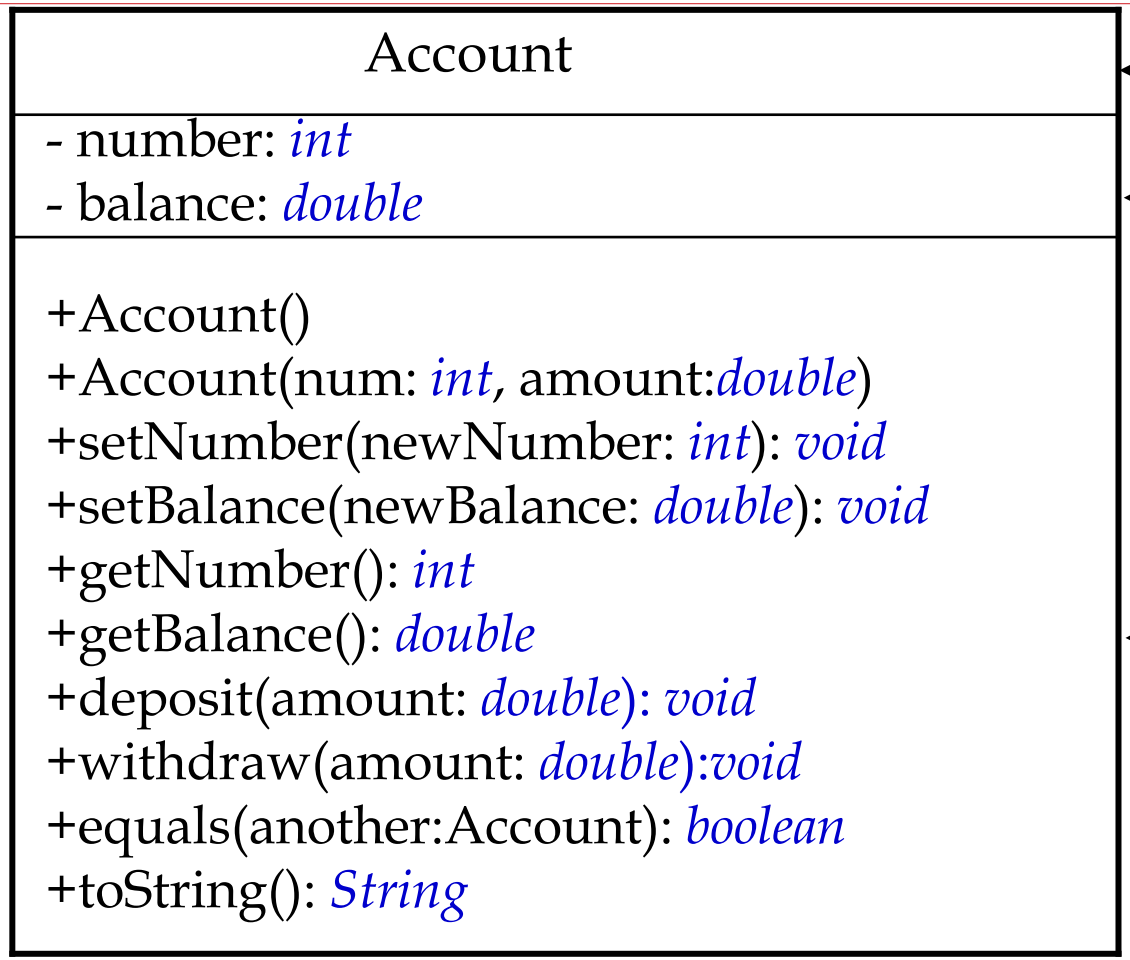
- **trasferFunds**: void method, takes another account as a parameter and should transfer funds from one account to the other provided there is enough funds in the first account.

- Modify **equals** method: two Account objects are now equal when they have identical account numbers and their balances are equal
- Test the new methods thoroughly.
  - Correct errors if any.

- Modify AccountTester program so that it performs the following:
  - Create two accounts: 21347 with a balance of P38763.87, and 61342 with a balance of P0.00.
  - Deposit some amount, read from the keyboard, into the second account.
  - Display the message "equal" if the two accounts are equal, otherwise "Not equal".
  - Withdraw P1000.00 from the first account, and display the message "overdrawn!" if the account ends up with a negative balance.

- Display the message ”Man! You are rich” if the first account has more than P20 000, otherwise display the message “Warning: You need to safe money!”.
- Prompt the user to specify the amount to withdraw, and carry out the withdrawal transaction provided the account ends up with at least P1000 after the transaction, otherwise display message “Account has insufficient funds.”

9. Drawing a UML class diagram



- − means private members
- + means public members

10. Method Signatures

- Used to differentiate between methods with the same name.

  Method Signature: Comprises of:

  a) method name

  b) data types in the formal parameter list.

- Examples

Method definition	Method signature
public int computeArea()	computeArea()
public int square(int x)	square(int)
public String xyz(int x, double y)	xyz(int,double)
public void abc(double y, int x)	abc(double, int)

11. Method Overloading

Method Overloading:
Having more than one method with the same name in a class
E.g.: Constructors, substring (String)

- **Overloaded** methods MUST have different signatures.
- Examples:
  - These methods are allowed in the same class:
    - public void ABC(int y, double c)
    - public int ABC(double x, int v)
  - However, these methods CANNOT be in the same class
    - public void ABC(int y, double c)
    - public int ABC(int x, double v)



Summary

- Objects overview
- Instantiating objects
- Java User-defined class structure
  - Instance variables
  - Methods (Constructors, void & value-returning methods)
- Writing User-defined class
- Testing user-defined class
- Parameter passing
- UML Class diagram
- Method signature and overloading



Next Lesson …

- Arrays

Q & A

